



LAB 3



JANUARY 30, 2025
Student Name: Madhvi Lashkaree
Student ID: 100997946

Lab 3

Note: You will be graded on the screenshot you attach of every step for proof of work. There are no data files to download for this chapter's projects, but create a Work\Chap05\Projects folder on your system before starting the projects.

In this project, you explore the MFT and learn how to locate date and time values in the metadata of a file you create. These steps help you identify previously deleted fragments of MFT records that you might find in unallocated disk space or in residual data in Pagefile.sys. You need the following for this project:

1. A system running Windows with the C drive formatted as NTFS
2. Notepad to create a small text file
3. WinHex (downloaded from <http://x-ways.net> and installed, if you haven't done so already) to analyze the metadata in the MFT

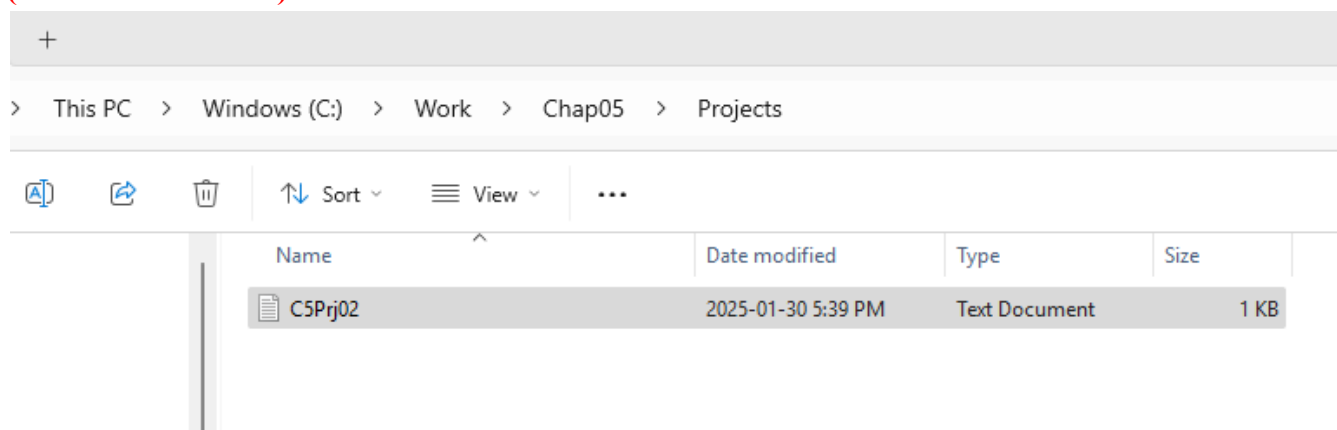
1. Start Notepad, and create a text file with one or more of the following lines:

- A countryman between two lawyers is like a fish between two cats.
- A slip of the foot you may soon recover, but a slip of the tongue you may never get over.
- An investment in knowledge always pays the best interest.
- Drive thy business or it will drive thee.

2. Save the file in your work folder as C5Prj02.txt and exit Notepad. (If your work folder isn't on the C drive, make sure you save the file on your C drive to have it entered in the

\$MFT files you copy later.)

(screenshot-Madhvi)



3. Next, review the material in “MFT and File Attributes,” paying particular attention to attributes 0x10 and 0x30 for file dates and times. The following charts show the offset byte count starting at position FILE of the file’s MFT record for the date and time stamps:

Note

The offsets listed in the following charts are from the first byte of the MFT record, not the starting position of the specific attributes 0x10 and 0x30.

0x10 \$Standard Information (data starts at offset 0x18)

Description of field	Offset position	Byte size
C Time (file creation)	0x50	8
A Time (file altered)	0x58	8
L Time (Last accessed)	0x60	8

0x30 \$File_Name (data starts at offset 0x18)

Description of field	Offset position	Byte size
C Time (file creation)	0xB8	8
A Time (file altered)	0xC0	8
R Time (file read)	0xC8	8
M Time (MFT change)	0xD0	8

Next, you examine the metadata of the `C5Prj02.txt` file stored in the \$MFT file. Follow these steps:

1. Start WinHex with the **Run as administrator** option. If you see an evaluation warning message, click **OK**.
2. As a safety precaution, click **Options, Edit Mode** from the menu. In the Select Mode (globally) dialog box, click **Read-only Mode (=write protected)**, as shown in *Figure 1*, and then click **OK**.

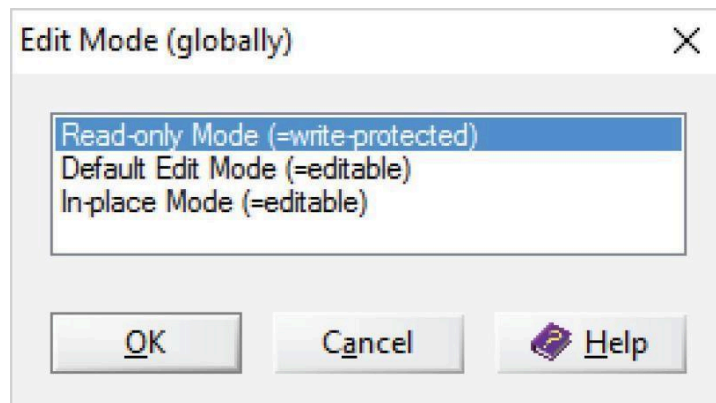
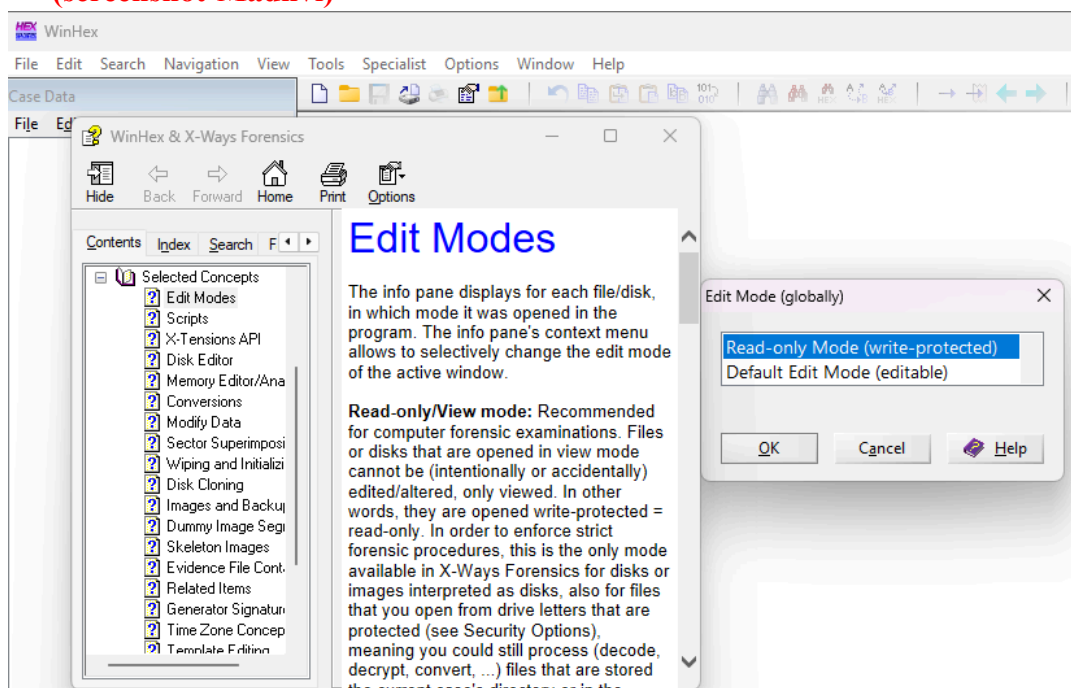


Figure 1: Changing WinHex to Read-Only Mode

Note

WinHex defaults to an editable mode, which means you can alter data in important system files and possibly corrupt them. When using a disk editor such as WinHex, always set it to read-only mode, unless you need to make specific modifications to data.

(screenshot-Madhvi)



3. Click **Tools, Open Disk** from the menu. In the View Disk dialog box, click the **C:** drive (or the drive where you saved `C5Prj02.txt`), as shown in *Figure 2*, and then click **OK**. If you're prompted to take a new snapshot, click **Take new one**. Depending on the size and quantity of data on your disk, it might take several minutes for WinHex to traverse all the files and paths on your disk drive.

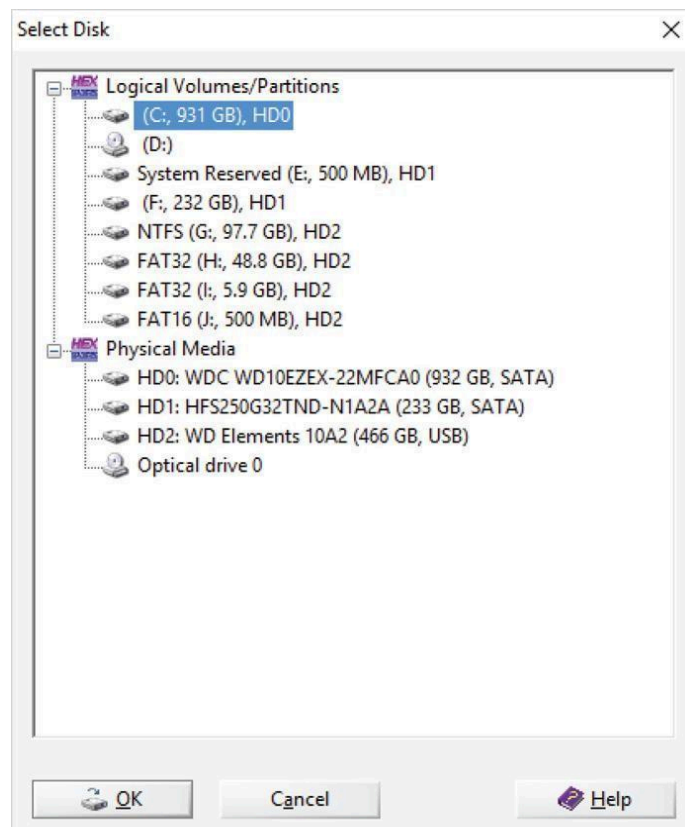
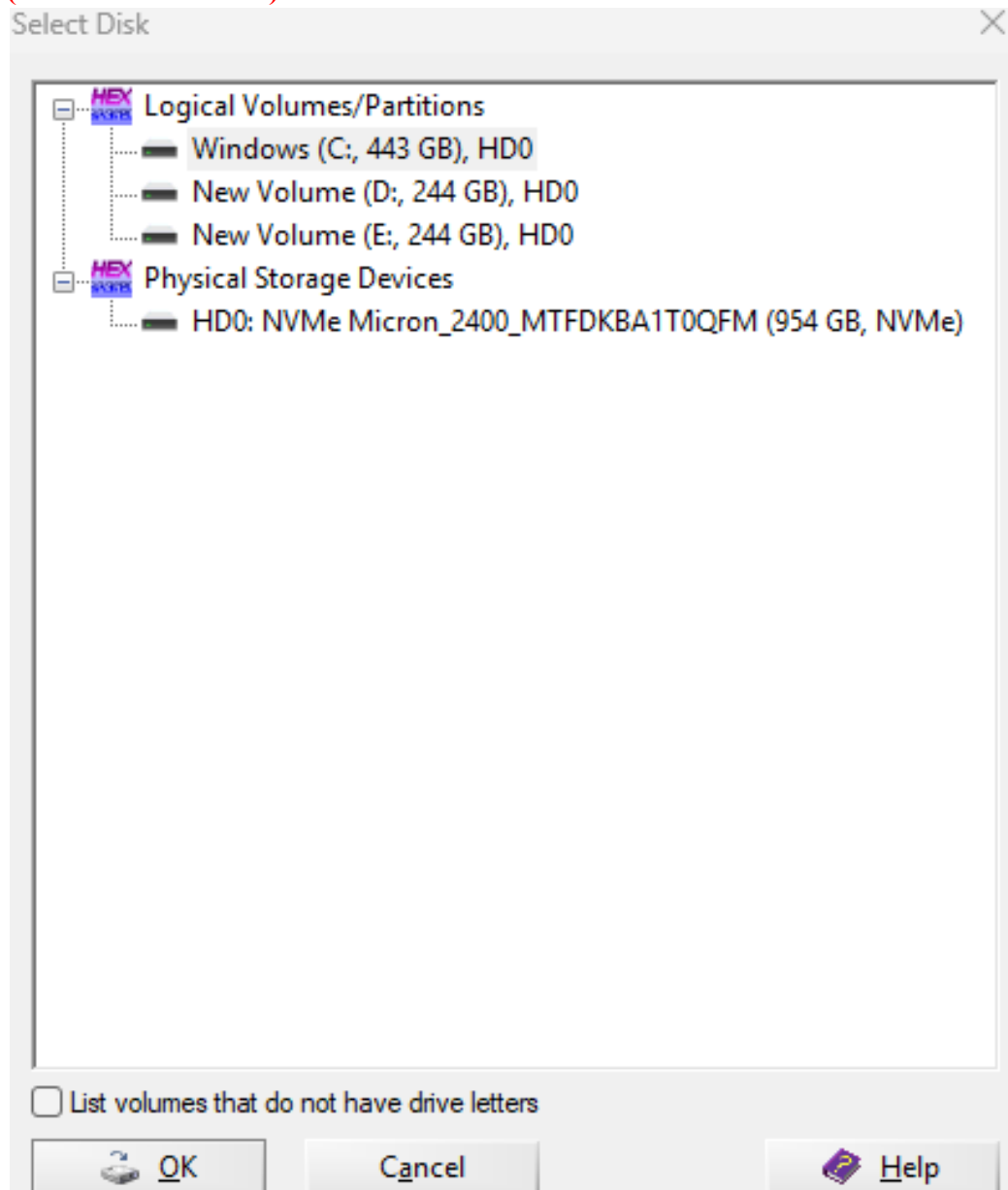


Figure 2: Selecting the Drive in WinHex

Note

By default, WinHex displays a floating Data Interpreter window that converts hex values to decimal values and can also convert date and time codes. If you don't see this window, activate it by clicking View, pointing to Show, and clicking Data Interpreter.

(screenshot-Madhvi)



4. Click **Options, Data Interpreter** from the menu. In the Data Interpreter Options dialog box, click the **Windows FILETIME (64 bit)** check box, shown in *Figure 3*, and then click **OK**. The Data Interpreter should then have FILETIME as an additional display item.

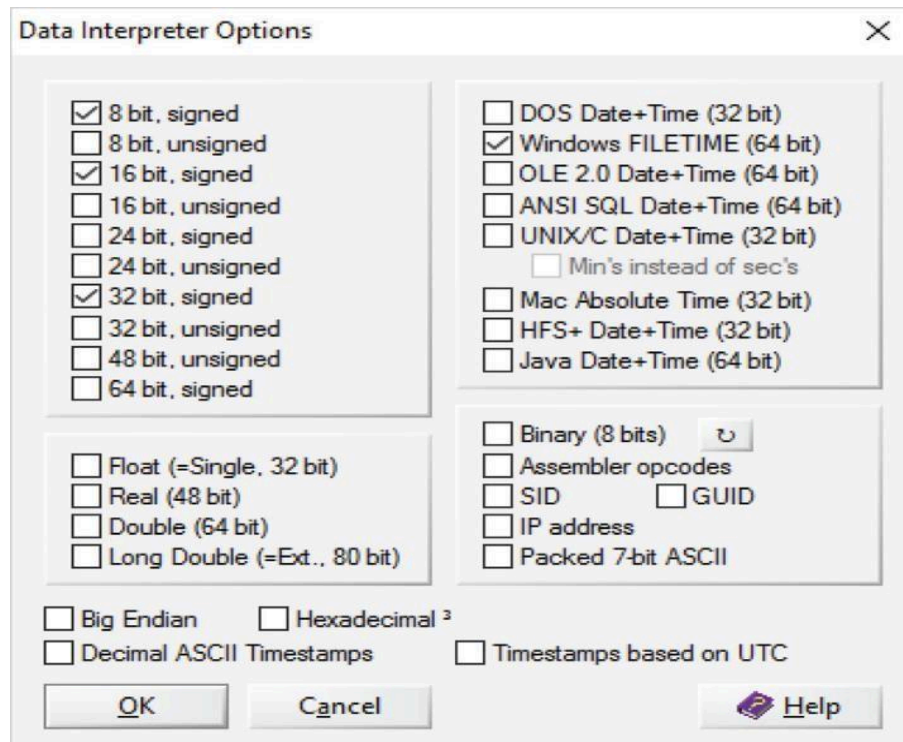
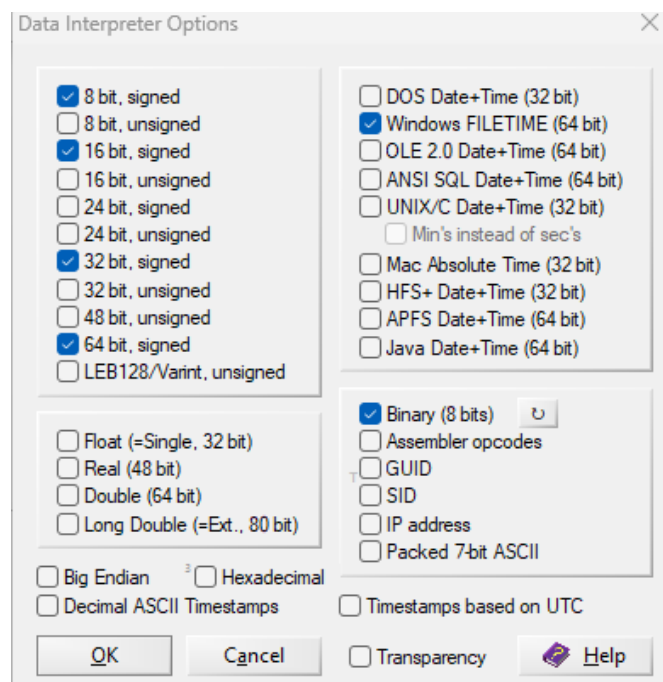


Figure 3: The Data Interpreter Options Dialog Box

(screenshot-Madhvi)



- Now you need to navigate to your work folder (C:\Work\Chap05\Projects) in WinHex. In the upper-right pane of WinHex, scroll down until you see your work folder. Double-click each folder in the path (see *Figure 4*), and then click the C5Prj02.txt file.

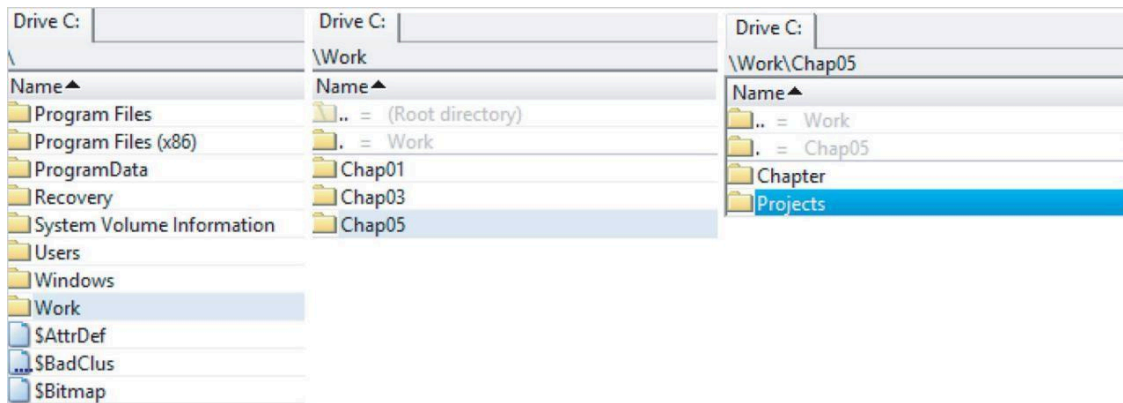


Figure 4: Navigating Through Folders in WinHex

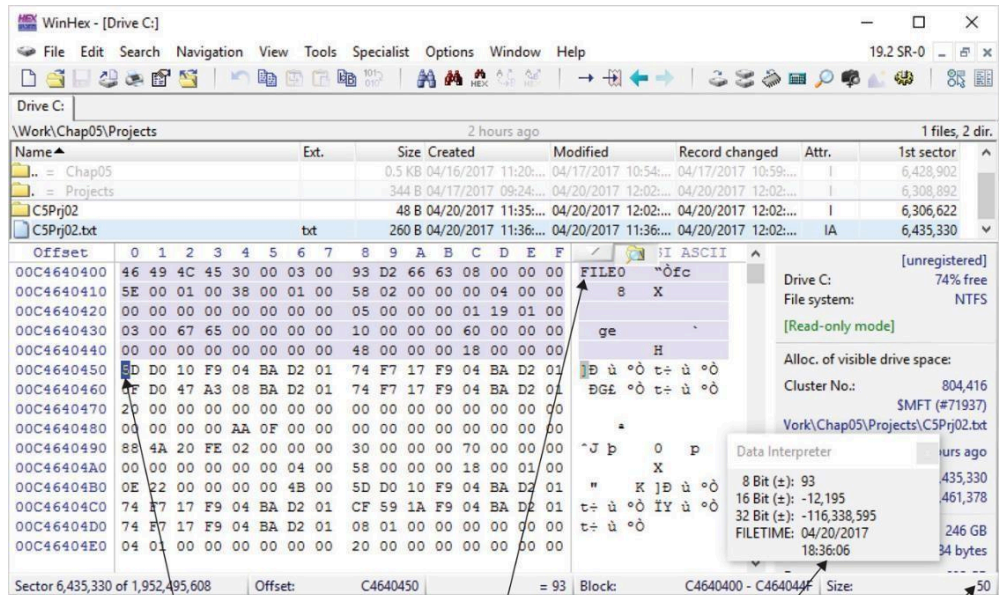
(screenshot-Madhvi)

Drive C: | C5Prj02.txt | C5Prj02.txt

\\Work\Chap05\Projects54 min. ago

Name▲	Ext.	Size	Created	Modified	Record changed	Attr.	1st sector
.. = Chap05		152 B	2025-02-02 14:17:07	2025-02-02 14:17:35	2025-02-02 14:17:35	I	6,533,206
. = Projects		152 B	2025-02-02 14:17:28	2025-02-02 14:18:00	2025-02-02 14:18:00	I	6,521,288
C5Prj02.txt	txt	264 B	2025-02-02 14:17:44	2025-02-02 14:18:22	2025-02-02 14:18:22	AI	6,302,340

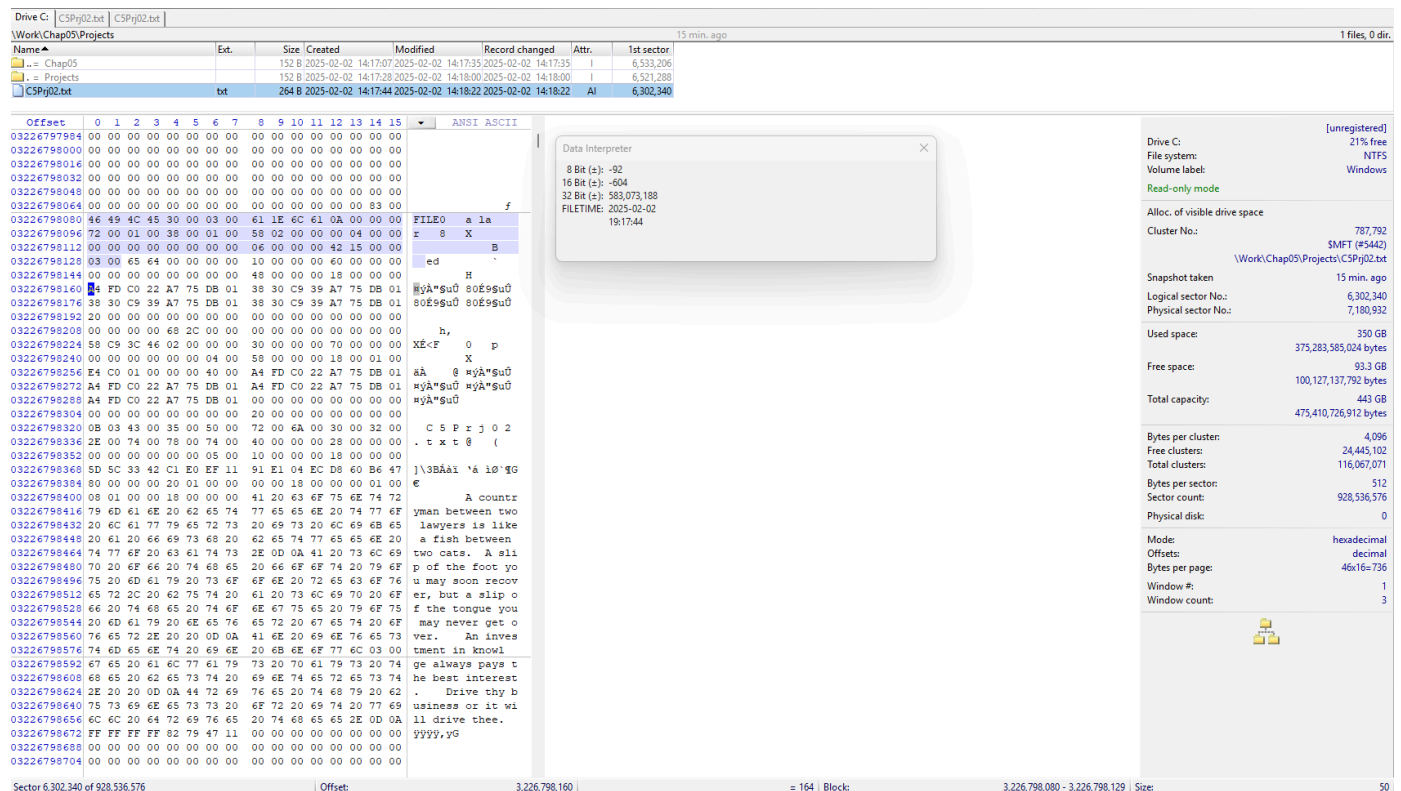
6. Drag from the beginning of the record, on the letter **F** in **FILE**, and then down and to the right while you monitor the hexadecimal counter in the lower-right corner. (Note: 50 hexadecimal bytes is the “position” for the first date and time stamp for this record, as described in the previous charts for 0x10 \$Standard Information.) When the counter reaches 50 (see *Figure 5*), release the mouse button.



After dragging, release mouse button and click here to interpret date and time Click here and drag down until offset counter shows 50 bytes Note date and time Offset counter

Figure 5: Locating the Date and Time Value in the MFT Record

(screenshot-Madhvi)



7. Move the cursor one position to the next byte (down one line and to the left), and record the date and time of the Data Interpreter's FILETIME values.

(screenshot-Madhvi)

The screenshot shows a file explorer window with the following structure:

- Drive C: | CSPj02.txt | CSPj02.txt | 15 min. ago | 1 files, 0 dir.
- \\Work\Chap05\Projects
- Name | Ext. | Size | Created | Modified | Record changed | Attr. | 1st sector
- .. | | 152 B | 2025-02-02 14:17:07 | 2025-02-02 14:17:35 | 2025-02-02 14:17:35 | I | 6,533,206
- .. | | 152 B | 2025-02-02 14:17:28 | 2025-02-02 14:18:00 | 2025-02-02 14:18:00 | I | 6,521,288
- CSPj02.txt | txt | 264 B | 2025-02-02 14:17:44 | 2025-02-02 14:18:22 | 2025-02-02 14:18:22 | AI | 6,302,340

Below the file list is a hex editor view showing memory offsets and ASCII values. A Data Interpreter window is open, displaying file metadata for CSPj02.txt, including file size, creation time, and file type.

8. Reposition the mouse cursor on the remaining offsets listed in the previous charts, and record their values.

(screenshot-Madhvi)

The screenshot shows a file explorer window with the following structure:

- \\Work\Chap05\Projects
- Name | Ext. | Size | Created | Modified | Record changed | Attr. | 1st sector
- .. | | 152 B | 2025-02-02 14:17:07 | 2025-02-02 14:17:35 | 2025-02-02 14:17:35 | I | 6,533,206
- .. | | 152 B | 2025-02-02 14:17:28 | 2025-02-02 14:18:00 | 2025-02-02 14:18:00 | I | 6,521,288
- CSPj02.txt | txt | 264 B | 2025-02-02 14:17:44 | 2025-02-02 14:18:22 | 2025-02-02 14:18:22 | AI | 6,302,340

Below the file list is a hex editor view showing memory offsets and ASCII values. A Data Interpreter window is open, displaying file metadata for CSPj02.txt, including file size, creation time, and file type.

(screenshot-Madhvi)

(screenshot-Madhvi)

(screenshot-Madhvi)

WinHex screenshot showing a file system view of a FAT file system. The file list includes 'Chap05', 'Projects', and 'CSPj02.txt'. The main window displays the hex dump of 'CSPj02.txt' with the ASCII column showing a message in Arabic. A 'Data Interpreter' window is open, displaying the message in Arabic. The right sidebar shows file system details for Drive C:.

Drive C: [unregistered] 21% free
File system: NTFS
Volume label: Windows
Read-only mode

Alloc. of visible drive space
Cluster No.: 787,792
SMFT (#5442)
\\Work\Chap05\Projects\CSPj02.txt

Snapshot taken: 15 min. ago
Logical sector No.: 6,302,340
Physical sector No.: 7,180,932

Used space: 350 GB
375,959,474,176 bytes

Free space: 92.6 GB
99,451,248,640 bytes

Total capacity: 443 GB
475,410,726,912 bytes

Bytes per cluster: 4,096
Free clusters: 24,280,090
Total clusters: 116,067,071

Bytes per sector: 512
Sector count: 928,536,576

Physical disk: 0

Mode: hexadecimal
Offsets: hexadecimal
Bytes per page: 46x16:736

Window #: 1
Window count: 5

(screenshot-Madhvi)

WinHex screenshot showing a file system view of a FAT file system. The file list includes 'Chap05', 'Projects', and 'CSPj02.txt'. The main window displays the hex dump of 'CSPj02.txt' with the ASCII column showing a message in Arabic. A 'Data Interpreter' window is open, displaying the message in Arabic. The right sidebar shows file system details for Drive C:.

Drive C: [unregistered] 21% free
File system: NTFS
Volume label: Windows
Read-only mode

Alloc. of visible drive space
Cluster No.: 787,792
SMFT (#5442)
\\Work\Chap05\Projects\CSPj02.txt

Snapshot taken: 15 min. ago
Logical sector No.: 6,302,340
Physical sector No.: 7,180,932

Used space: 350 GB
375,959,474,176 bytes

Free space: 92.6 GB
99,451,248,640 bytes

Total capacity: 443 GB
475,410,726,912 bytes

Bytes per cluster: 4,096
Free clusters: 24,280,090
Total clusters: 116,067,071

Bytes per sector: 512
Sector count: 928,536,576

Physical disk: 0

Mode: hexadecimal
Offsets: hexadecimal
Bytes per page: 46x16:736

Window #: 1
Window count: 5

9. When you're finished, exit WinHex and hand in the date and time values you recorded.

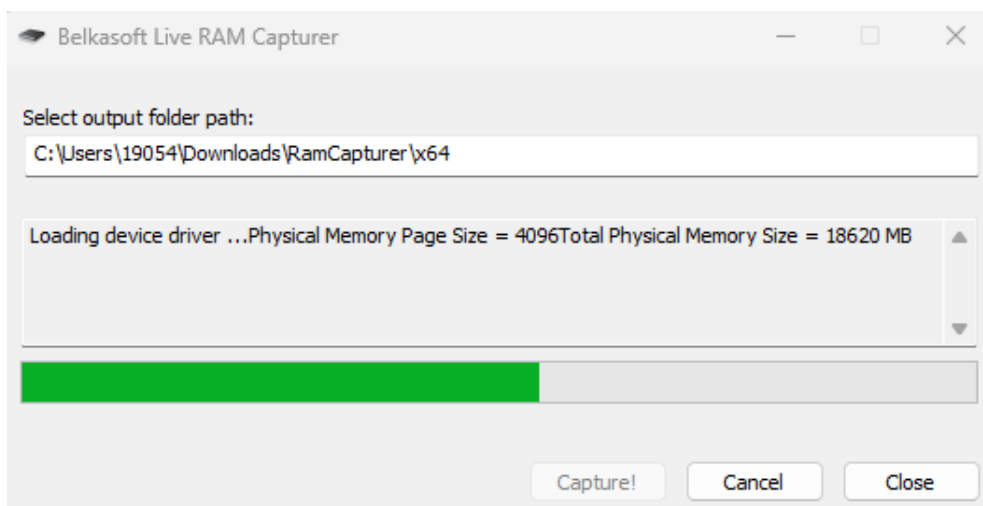
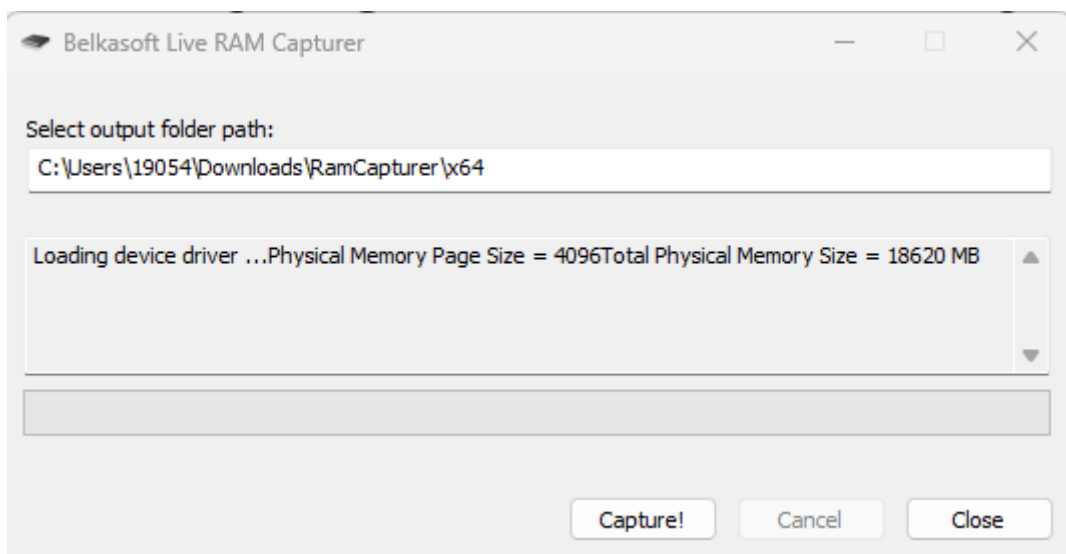
Lab 6 (Madhvi_100997946)

The objective of these projects is to give you practice in mounting and imaging virtual machines. Before beginning, create a *Work\Lab_6\Projects* folder on your system.

Note: You will be graded on the screenshot you attach of every step and your final report on it for proof of work.

Task: In this project, you install and use a tool for capturing live RAM. Follow these steps:

1. Start a Web browser, go to <https://belkasoft.com/ram-capturer>, and download *Belkasoft Live RAM Capturer*. Install this tool, accepting the defaults, and then start it.
2. Take a screenshot of the opening window. In the “Select output folder path” text box, type your work folder name, and then click **Capture!** When the capture is finished, click **Close**.



3. Using WinHex, the hex editor you've used in previous labs, open the testdump file and examine its contents. Do a search for keywords you'd expect to find in a physical machine's RAM, such as Microsoft, From, To, Subject, and http. If you find other keywords, make note of them. Take screenshots of the search results.

The screenshot shows the WinHex interface with a search for 'Microsoft'. The search results are displayed in the main window, showing the offset 00006B7790 and the text 'Microsoft'. The search dialog box is open, showing the search criteria and options.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ANSI ASCII
00006B76D0	38	66	62	64	64	62	61	32	2E	78	6D	6C	56	65	72	73	8fbddba2.xmlVers
00006B76E0	69	6F	6E	00	31	31	36	64	61	34	38	38	35	30	32	63	ion 116da488502c
00006B76F0	39	36	35	31	31	37	33	36	39	37	30	62	33	30	34	61	96511736970b304a
00006B7700	37	65	32	34	30	66	63	30	62	61	38	62	2E	78	6D	6C	7e240fc0ba8b.xml
00006B7710	38	31	64	35	63	36	30	63	64	61	61	32	66	36	62	61	81d5c60cdaa2f6ba
00006B7720	63	36	30	30	34	35	32	63	31	32	65	61	63	39	31	65	c600452c12eac91e
00006B7730	33	38	32	33	64	39	65	66	2E	78	6D	6C	00	00	00	00	3823d9ef.xml
00006B7740	0C	00	00	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	YYYY
00006B7750	FF	FF	FF	FF	00	00	00	00	61	6D	64	36	34	00	00	00	YYYY amd64
00006B7760	00	00	00	00	19	00	00	00	00	00	00	00	00	00	00	00	YYYY
00006B7770	00	00	00	00	FF	FF	FF	FF	01	00	00	00	00	00	00	00	YYYYYYYY
00006B7780	2E	00	00	00	00	00	00	00	00	00	00	00	FF	FF	FF	FF	. YYYY
00006B7790	FF	FF	FF	FF	01	00	00	00	4D	69	63	72	6F	73	6F	66	YYYY Microsoft
00006B77A0	74	2D	57	69	6E	64	6F	77	73	2D	57	69	6E	64	6F	77	t-Windows-Window
00006B77B0	73	55	70	64	61	74	65	43	6C	69	65	6E	74	2D	43	6F	sUpdateClient-Co
00006B77C0	72	65	00	00	00	00	00	00	2B	00	00	00	00	00	00	00	re +
00006B77D0	00	00	00	00	FF	FF	FF	FF	01	00	00	00	00	00	00	00	YYYYYYYY
00006B77E0	37	31	30	36	66	65	38	63	65	30	62	33	33	63	66	64	7106fe8ce0b33cfd
00006B77F0	66	65	65	38	62	35	38	65	37	65	31	62	30	61	38	31	fee8b58e7elb0a81
00006B7800	32	39	33	35	39	30	35	65	2E	78	6D	6C	35	36	61	37	2935905e.xml56a7
00006B7810	64	35	30	37	64	30	36	66	33	64	35	31	38	32	31	65	d507d06f3d51821e
00006B7820	31	31	32	34	65	36	34	32	66	61	37	34	66	66	33	33	1124e642fa74ff33
00006B7830	39	37	61	66	2E	78	6D	6C	50	72	6F	63	65	73	73	6F	97af.xmlProcesso
00006B7840	72	41	72	63	68	69	74	65	63	74	75	72	65	00	00	00	rArchitecture
00006B7850	00	00	00	00	20	00	00	00	00	00	00	00	00	00	00	00	

The screenshot shows the WinHex interface with a search for 'from'. The search results are displayed in the main window, showing the offset 00006B9E10 and the text 'from'. The search dialog box is open, showing the search criteria and options.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	ANSI ASCII
00006B9D40	67	73	20	73	6F	63	6B	65	74	46	6C	61	67	73	2C	20	gs socketFlags,
00006B9D50	45	6E	64	50	6F	69	6E	74	20	72	65	6D	6F	74	65	45	EndPoint remoteE
00006B9D60	50	29	0D	0A	20	20	61		74	20	41	50	49	5F	44	79	P) at API_Dy
00006B9D70	6E	61	6D	69	63	2E	53	53	44	50	5F	4F	62	6A	2E	73	namic.SSDP_Obj.s
00006B9D80	74	61	72	74	51	75	65	72	79	28	29	0D	0A	32	30	32	tartQuery() 202
00006B9D90	34	2D	30	38	2D	32	33	20	31	30	3A	34	33	3A	34	35	4-08-23 10:43:45
00006B9DA0	2E	31	36	31	20	20	20	20	20	20	41	50	49	20	43	68	.161 API Ch
00006B9DB0	65	63	6B	20	79	20	54	79	70	65	3A	6E	62	0D	0A	32	eck y Type:nb 2
00006B9DC0	30	32	34	2D	30	38	2D	32	33	20	31	30	3A	34	33	3A	024-08-23 10:43:
00006B9DD0	34	37	2E	31	35	35	20	20	20	20	20	20	53	74	6F	70	47.155 Stop
00006B9DE0	20	61	6C	6C	20	71	75	65	72	79	0D	0A	32	30	32	34	all query 2024
00006B9DF0	2D	30	38	2D	32	33	20	31	30	3A	34	33	3A	34	37	2E	-08-23 10:43:47.
00006B9E00	31	35	35	20	20	20	20	20	20	4C	6F	61	64	20	66	72	155 Load fr
00006B9E10	6F	6D	20	4D	79	44	65	76	69	63	65	0D	0A	32	30	32	om MyDevice 202
00006B9E20	34	2D	30	38	2D	32	33	20	31	30	3A	34	33	3A	34	37	4-08-23 10:43:47
00006B9E30	2E	31	37	30	20	20	20	20	20	20	72	65	73	74	6F	72	.170 restor
00006B9E40	60	0D	0A	32	30	32	34	2D	30	38	2D	32	33	20	61	30	e 2024-08-23 10
00006B9E50	3A	34	33	3A	34	37	2E	31	38	36	20	20	20	20	20	20	:43:47.186
00006B9E60	51	75	65	72	79	44	6F	6E	65	0D	0A	32	30	32	34	2D	QueryDone 2024-
00006B9E70	30	38	2D	32	33	20	31	30	3A	34	33	3A	34	37	2E	31	08-23 10:43:47.1
00006B9E80	38	36	20	20	20	20	20	20	4E	6F	6E	65	20	52	6F	75	86 None Rou
00006B9E90	74	65	72	20	44	65	76	69	63	65	0D	0A	32	30	32	34	ter Device 2024
00006B9EA0	2D	30	38	2D	32	33	20	31	30	3A	34	33	3A	34	37	2E	-08-23 10:43:47.
00006B9EB0	31	38	36	20	20	20	20	20	20	45	76	65	6E	74	52	65	186 EventRe

Del to restart



https://



Copyright License
e. Subject to the
terms and conditions of



4.Exit WinHex. Write a short report on your findings and describe the challenges and possibilities of using Belkasoft for capturing live RAM and WinHex for analysis in an actual investigation.

Lab 6 Project Report: Capturing and Analyzing Live RAM

This project aimed to provide hands-on experience in capturing live RAM using Belkasoft Live RAM Capturer and analyzing the captured data with WinHex. The exercise simulates a real-world digital forensics investigation where volatile memory can contain critical evidence such as encryption keys, active processes, and open files.

Findings and Observations

RAM Capture

1. The RAM capture was successfully performed and stored as 20250317.mem in the designated output folder.
2. The process was fast and completed without errors.

Analysis Using WinHex

1. **Microsoft:** Multiple occurrences found, likely associated with active Windows processes.
2. **From:** Indicated email-related data stored in RAM.
3. **To:** Found alongside From, confirming the presence of email communications.
4. **Subject:** Located in email headers, suggesting that email content was retained in memory.
5. **http:** Detected within URLs, providing evidence of web browsing activities.

Challenges Encountered

1. The RAM dump was several gigabytes in size, making it time-consuming to load and analyze in WinHex.
2. Not all extracted data was in readable plain text, requiring knowledge of hexadecimal interpretation.
3. While keyword searches yielded results, understanding the context of the data required additional forensic tools.

Practical Applications

1. RAM capture can help identify running malicious processes or memory-resident malware.
2. RAM dumps can provide crucial evidence, such as encryption keys, open files, and active network connections.
3. Captured data can reveal browsing history, active logins, and email exchanges.

Belkasoft Live RAM Capturer and WinHex are powerful tools for digital forensic investigations, allowing analysts to recover valuable volatile data before it is lost. While capturing RAM is efficient, analyzing the data presents challenges due to its large size and the need for expertise in hexadecimal interpretation. Nonetheless, RAM analysis remains a crucial aspect of incident response and forensic investigations, providing insights into system activities and potential security threats.

